SLHA v2: Cache-Aware Hybrid Attention with Low-Rank Latents and 1-Bit Sign-LSH Residuals for Memory-Bandwidth-Bound LLM Inference

Tarek Zekriti*
Independent Researcher

June 2026

Abstract

Autoregressive inference of large language models (LLMs) is bounded by memory bandwidth: the key/value (KV) cache grows linearly with context length and saturates the interconnect long before the arithmetic units are busy. We present **SLHA v2**, a cache-aware attention mechanism that decomposes each key into (i) a shared low-rank latent base quantized to 4 bits and (ii) a deterministic 1-bit sign-LSH residual. The two are combined in a single *fused* score that adds a continuous dot product to a branchless Hamming-distance correction whose exactness we prove. The representation is packed into a hardware-aligned **128-byte tile with zero padding** that maps onto cache lines, and is managed by an elastic KV cache (*CCOS Soft-Paging*) whose three-state HOT/WARM/COLD machine bounds the memory footprint under a byte budget *without I/O*, by freeing residuals where they matter least. We provide an open, zero-dependency Rust reference implementation (SciRust) with runtime-dispatched scalar / AVX2 / AVX-512 / NEON kernels proven equivalent by 37 automated tests. On reproducible synthetic benchmarks (fixed seeds) we measure: a 1-bit core that exactly realizes the signed-dot identity; an attention-*output* cosine fidelity of 0.95–0.997 versus full precision, *despite* score-ranking Spearman plateauing at 0.79–0.90 (the softmax absorbs score error); $\sim$2.5$\times$ more scored tokens/s than a `bf16` baseline at comparable memory bandwidth; SIMD speedups of 11.5$\times$ (AVX2) and 14.1$\times$ (AVX-512); and a learned, task-aware low-rank projection that lifts WARM Spearman from 0.16 to 0.86 under query/key distribution shift. A controlled ablation shows that quantization width is *not* the bottleneck (an INT8 reference does not beat INT4 on the coarse term)—the low-rank projection is. We position SLHA v2 as a validated *mechanism* and a systems substrate, and we are explicit about what remains unmeasured: real-model perplexity, hardware cache counters, and joint projection training.

Keywords: efficient inference, KV cache, low-rank attention, 1-bit quantization, locality-sensitive hashing, cache-aware data structures, SIMD.

1 Introduction

Conventional attention pipelines treat the KV cache as a contiguous stream of high-precision tensors (FP16/BF16). During decoding, each new token re-reads the entire history from main memory into the registers of the CPU/GPU. The cost of this re-read is set by the cache hierarchy:

- **L1 (data):** $\sim$1 cycle, but tiny (32–64 KiB/core);

*Independent Researcher. Correspondence: `zekrititarek@gmail.com`.

- **L2:** ∼4–15 cycles, 0.5–2 MiB/core;
- **L3 / LLC:** shared, ∼40–80 cycles, tens to hundreds of MiB.

If the KV layout does not align with these strata, the processor spends most of its time waiting on transfers (cache misses) rather than computing. This is the *memory-bandwidth wall*: for long contexts, arithmetic intensity collapses and FLOPs become free relative to bytes.

SLHA v2 attacks the wall at the level of the data structure itself. Its contributions are:

1. **An asymmetric key representation** (§3): a shared low-rank latent (4-bit) plus a deterministic 1-bit sign-LSH residual, fused into one score that combines a float dot product with a Hamming correction. We prove the binary core is an exact signed dot product (Lemma 1).
2. **A hardware-aligned 128-byte tile** (§4) with *zero padding*, whose 64-byte latent sub-block is exactly one cache line.
3. **An elastic KV cache with Soft-Paging** (§5): a HOT/WARM/COLD state machine that bounds the footprint under a byte budget by masking residuals ($O(1)$, no I/O, no allocation) before evicting the most causally distant tokens.
4. **An open, zero-dependency reference implementation** (§6) with runtime-dispatched SIMD kernels proven equivalent to a scalar reference.
5. **A reproducible, deliberately honest evaluation** (§7) that separates what is *measured* from what is *targeted*, including negative results (§8).

A guiding finding, established by ablation (§7.7), is that the fidelity ceiling of the coarse term is set by the *low-rank projection*, not by latent quantization width: doubling the bits (INT8) does not help, while a *learned* projection lifts quality dramatically (§7.4). This reframes where future effort should go.

Scope statement. This paper reports the validation of a *mechanism* on synthetic data with reproducible, seeded benchmarks. We do not claim end-to-end perplexity on a real model; §9 states the boundaries precisely. We believe an honest account of a partially validated system is more useful than an overclaimed one.

2 Background and Related Work

Low-rank / latent KV compression. Multi-head Latent Attention (MLA) in DeepSeek-V2 [2] projects keys and values through a shared low-rank bottleneck to shrink the KV cache. SLHA v2 adopts the same low-rank *base* but adds a quantized 1-bit residual to recover the information the bottleneck discards, and ties the layout to the cache line.

KV-cache quantization. GPTQ [8], AWQ [9], and KIVI [11] quantize weights or KV entries to low bit-width. NF4 (from QLoRA [7]) uses a normal-float codebook. SLHA v2 uses signed INT4 (with optional grouped micro-scaling and NF4) for the latent, but our ablation (§7.7) shows quantization width is not the limiting factor here.

1-bit and salient-aware quantization. BiLLM [10] pushes post-training quantization toward 1 bit by isolating salient weights in higher precision. SLHA v2's residual is 1-bit *by construction* (a sign-LSH code), and §10 discusses a BiLLM-style salient extension as future work.

Locality-sensitive hashing. Sign-LSH / SimHash [4] estimates the angle between two vectors from the agreement of random hyperplane signs; the underlying random projection is justified by the Johnson–Lindenstrauss lemma [5]. SLHA v2's residual is a sign-LSH code of the reconstruction error, and the fused score reads it back via a Hamming distance.

KV eviction and paging. H2O [12] and StreamingLLM [13] drop or retain KV entries by heuristics; PagedAttention/vLLM [3] manages KV memory in pages. SLHA v2's Soft-Paging adds an intermediate, near-lossless WARM state—freeing only the 1-bit residual—*before* resorting to eviction, and chooses what to free by residual energy.

3 The SLHA v2 Mechanism

3.1 Asymmetric decomposition

SLHA v2 splits the representation of each key into two asymmetric components: a compact semantic base and a high-frequency binary correction. Let $x_j \in \mathbb{R}^{d_{\text{model}}}$ be the activation of context token j.

3.2 Multi-head latent compression

A shared down-projection maps x_j into a latent bottleneck of dimension d_c:

$$h_{KV,j} \;=\; W_{\text{down}}\, x_j \;\in\; \mathbb{R}^{d_c}. \tag{1}$$

Each head n reconstructs a *coarse key* via a head-specific up-projection $W^{(n)}_{\text{up},K} \in \mathbb{R}^{d_k \times d_c}$:

$$K^{(n)}_{\text{coarse},j} \;=\; W^{(n)}_{\text{up},K}\, h_{KV,j}. \tag{2}$$

3.3 Deterministic 1-bit sign-LSH residual

The reconstruction error $E^{(n)}_j = K^{(n)}_{\text{real},j} - K^{(n)}_{\text{coarse},j}$ is captured by a fixed random orthogonal Johnson–Lindenstrauss projection $Z \in \mathbb{R}^{d_s \times d_k}$, keeping only the sign on d_s dimensions:

$$B^{(n)}_j \;=\; \text{sign}\Big(Z\, E^{(n)}_j\Big) \;\in\; \{-1,+1\}^{d_s}. \tag{3}$$

Z is stable and pseudo-random (seeded at start-up); $B^{(n)}_j$ is stored as compact bitmaps. With $d_s = 256$ the residual occupies exactly 32 bytes (four 64-bit words).

3.4 Fused float–binary score

The unnormalized attention score between query $Q^{(n)}_i$ and cached token j combines a continuous inner product and a binary inner product accelerated by bitwise logic:

$$s^{(n)}_{ij} \;=\; \underbrace{\langle Q^{(n)}_i W^{(n)}_{\text{up},K}, h_{KV,j}\rangle}_{\text{coarse (continuous)}} \;+\; \lambda \cdot \underbrace{\Big[\, d_s - 2\,\text{popcount}\big(p(Q^{(n)}_i) \oplus B^{(n)}_j\big)\Big]}_{\text{residual (1-bit)}}, \tag{4}$$

where $\oplus$ is bitwise XOR, popcount counts set bits, and $p(Q^{(n)}_i) = \text{pack\_sign}(Z\, Q^{(n)}_i)$ packs the query's sign-LSH code into the same bitmap format. The next lemma shows the bracketed term is an *exact* signed dot product, not an approximation of one.

Table 1: The `SciRustSlhaTile` layout ($d_c = 128$, $d_s = 256$). Total $= 64 + 32 + 32 = 128$ bytes; `align_of` is 128 on AArch64 and 64 elsewhere (size is 128 either way); verified by test `tile_is_exactly_128_bytes_zero_padding` and by AArch64 cross-compilation.

Field	Type	Bytes	Role
`latent_kv`	`[u8; 64]`	64	128 INT4/NF4 latent dims (2/byte)
`residual_bitmap`	`[u64; 4]`	32	sign-LSH residual (256 bits)
`scale`	`f32`	4	global INT4 dequant scale
`dynamic_lambda`	`f32`	4	residual weight λ (Eq. 6)
`residual_sigma`	`f32`	4	per-tile σ_E (for λ)
`token_id`	`u32`	4	token identifier
`position`	`u32`	4	causal position
`head_id`	`u16`	2	attention head
`flags`	`u16`	2	HOT/WARM/NF4
`group_scales`	`[u8; 8]`	8	per-group MX micro-scales

Lemma 1 (Hamming identity)**.** *Let $a, b \in \{0,1\}^{d_s}$ encode signs via $\sigma(0) = +1$, $\sigma(1) = -1$, and let $s_a, s_b \in \{-1,+1\}^{d_s}$ be the corresponding sign vectors. Then*

$$d_s - 2\,\mathrm{popcount}(a \oplus b) \;=\; \sum_{k=1}^{d_s} s_{a,k}\, s_{b,k} \;=\; \langle s_a, s_b \rangle. \tag{5}$$

Proof. Bit k of $a \oplus b$ is 1 iff $a_k \neq b_k$, i.e. iff $s_{a,k} s_{b,k} = -1$. Let $D = \mathrm{popcount}(a \oplus b)$ be the number of disagreements. Agreements contribute $+1$ and disagreements -1, so $\langle s_a, s_b \rangle = (d_s - D) \cdot (+1) + D \cdot (-1) = d_s - 2D$. □

This identity is verified against a brute-force reference for 2000 random bitmap pairs (test `hamming_identity_matches_sign_dot`, §7). It is what makes the residual term a single XOR followed by a population count—two of the cheapest operations on any modern ISA.

3.5 Dynamic λ calibration

The weight λ of the binary correction is not fixed. From the residual energy $\sigma_E = \mathrm{RMS}(E_j^{(n)})$ the spec sets

$$\lambda \;=\; \sigma_E \cdot \sqrt{\tfrac{\pi}{2\,d_s}}. \tag{6}$$

The *shape* $\lambda \propto \sigma_E$ is validated empirically (§7.8): the closed-form optimal multiplier against a full-precision reference is near-constant across residual energy. The analytic *constant* $\sqrt{\pi/(2d_s)} \approx 0.078$ at $d_s = 256$ under-weights the residual by $\approx 4.2\times$; the calibrated constant is $C_{\mathrm{emp}} \approx 0.33$. We keep the analytic form as the conservative default and document the calibrated constant (§7.8).

4 Hardware-Aware Tile Layout

4.1 The 128-byte tile

SLHA v2 stores each (key, residual, metadata) triple in a single `#[repr(C)]` structure whose total size is exactly 128 bytes *with zero padding*. Its alignment is *cache-line-aware* and resolved at compile time from the target architecture (§4.2). Table 1 gives the layout.

The signed INT4 dequantization uses a baked-in zero point, $\hat{v}_d = (\text{nibble}_d - 8) \cdot \texttt{scale}$, so the latent represents negative values without a separate zero-point array. Optional grouped micro-scaling (MX) stores one `u8` scale per group of 16 dims in `group_scales`, giving effective scale $\texttt{scale} \cdot \texttt{group\_scales}[g]/255$.

```rust
// Cache-line-aware alignment, resolved at compile time (size is 128 either way).
#[cfg_attr(target_arch = "aarch64", repr(C, align(128)))] // one 128B line
#[cfg_attr(not(target_arch = "aarch64"), repr(C, align(64)))] // two 64B lines
pub struct SciRustSlhaTile {
    pub latent_kv: [u8; 64], // INT4/NF4 latent (128 dims)
    pub residual_bitmap: [u64; 4], // sign-LSH residual (256 bits)
    pub scale: f32, pub dynamic_lambda: f32, pub residual_sigma: f32,
    pub token_id: u32, pub position: u32,
    pub head_id: u16, pub flags: u16,
    pub group_scales: [u8; 8],
}

// score = <q_coarse, dequant(latent)> + lambda * (d_s - 2*popcount(q_sign ^ B))
// Branchless Hamming reduction; one vpopcntq over 256 bits on AVX-512 VPOPCNTDQ,
// else count_ones() (-> POPCNT/CNT). Runtime-selected; bit-equivalent by test.
pub fn hamming_distance(a: &[u64; 4], b: &[u64; 4]) -> u32 {
    #[cfg(target_arch = "x86_64")]
    if is_x86_feature_detected!("avx512vpopcntdq") && is_x86_feature_detected!("avx512vl") {
        return unsafe { hamming_vpopcntdq(a, b) };
    }
    (0..4).map(|w| (a[w] ^ b[w]).count_ones()).sum()
}
```

Listing 1: Core tile and fused score (abridged from `scirust/src/attention/slha_v2.rs`).

4.2 Cache mapping

The 64-byte latent sub-block is exactly one cache line and holds all 128 semantic dimensions; it is this sub-block—not the whole tile—that feeds the arithmetic unit in a single line load. Queries are pre-loaded into L1 and the context tiles are swept sequentially, so the access pattern is a pure streaming read with unit stride.

Because cache-line width differs across our targets—64 bytes on mainstream x86-64, 128 bytes on Jetson Thor-class AArch64 (Neoverse-V3AE)—we resolve the alignment attribute at compile time from `target_arch` via `cfg_attr` (listing above): `align(128)` on AArch64, so a tile is *exactly one* cache line and a single line fill on the LPDDR5X bus; `align(64)` elsewhere, so a tile spans two lines. The *size* is 128 bytes in both cases (128 is a multiple of both alignments), so zero padding holds regardless of target. No `build.rs` is needed because `target_arch` is a compile-time `cfg`; a build script would only help to probe the *host's* actual line size under `-C target-cpu=native` (§10). We cross-compile to `aarch64-unknown-linux-gnu` in CI so the AArch64 layout and NEON kernel are type-checked.

5 CCOS Elastic KV-Cache and Soft-Paging

SLHA v2's decomposition lets the runtime apply a *boundedness* policy as a fine-grained elasticity of fidelity rather than all-or-nothing eviction. Each tile occupies one of three states (Table 2).

Table 2: Soft-Paging states. Footprint is the *logical* byte cost used by the budget.

State	Behavior	Footprint
HOT	full fused score (latent + 1-bit residual)	128 B
WARM	residual freed ($\lambda = 0$, `FLAG_WARM`); score = coarse only	96 B (-25%)
COLD	evicted from the working set; slot recycled on next insert	0 B

Table 3: Soft-Paging under a byte budget (8192 tiles, $d_v = 64$).

Regime	Budget	HOT/WARM/COLD	Footprint	cos(out, all-HOT)
A — paging only	896 KiB (112 B/tile)	4096 / 4096 / 0	896 KiB (88%)	**0.9995**
B — forced eviction	320 KiB (40 B/tile)	0 / 3413 / 256	319 KiB $\leq$ budget	—

5.1 State machine and budget enforcement

A reference manager, `ccos::ElasticKvCache`, holds tiles in a contiguous arena. `enforce_budget()` bounds the logical footprint under a byte budget in two phases:

1. **Page** HOT→WARM in `PageOutPolicy` order. The default *hybrid* policy pages the lowest-σ_E tiles first—their 1-bit residual matters least, so WARM is near-lossless there (§7.3). Masking is $O(1)$: zero the 32-byte bitmap and set a flag, no allocation, no I/O.
2. **Evict** live tiles →COLD if still over budget, *always oldest-first* (causal distance). Dropping a token is the harder loss, so it targets the most distant context regardless of σ_E. The freed slot is recycled via a free list.

This two-key design (page by energy, evict by age) is pinned by tests (§7). A production CCOS would snapshot COLD tiles to an append-only event log; we do not simulate that persistence here.

5.2 Soft-Paging fidelity

The example `ccos_softpaging` streams 8192 tiles (1024 KiB all-HOT) and measures the effect on the attention *output* (Table 3). Paging *half* the tiles—the lowest-σ_E ones—to WARM leaves the output at cosine ≈ 0.9995 versus all-HOT: the central benefit of Soft-Paging is to bound memory by freeing the residual where it weighs least, with no I/O and no dropped token.

6 Reference Implementation: SciRust

The kernel is written in Rust (2021 edition) with a no-allocation invariant in the hot loops. Salient engineering properties:

- **Runtime SIMD dispatch.** The coarse dot product (128-dim INT4 dequant + FMA) has scalar, AVX2, AVX-512 (x86_64) and NEON (aarch64) paths, selected at run time via `is_x86_feature_detected!` (order AVX-512 > AVX2 > scalar) with a portable fallback. There is *no* compile-time feature gating of correctness.
- **Branchless residual.** The Hamming term is a fixed four-word reduction with no data branches. It dispatches at run time to a single `vpopcntq` over all 256 residual bits (`_mm256_popcnt_epi64`) on x86-64 CPUs advertising AVX-512 VPOPCNTDQ+VL, otherwise to `count_ones()` (→ `POPCNT`/`CNT`). The two are asserted bit-equal on capable hardware and compile-checked elsewhere.

Table 4: Score rank fidelity vs. FP (ideal base). HOT = latent + residual; WARM = latent only.

ρ	HOT Spearman	HOT top-16	WARM Spearman	WARM top-16
0.05	0.987	0.875	0.987	0.875
0.10	0.983	0.750	0.982	0.750
0.20	0.966	0.750	0.961	0.688
0.30	0.937	0.625	0.925	0.562
0.50	0.844	0.500	0.811	0.438
0.70	0.702	0.375	0.627	0.250

- **Zero dependencies.** The library has no runtime dependencies; `criterion` is a dev-only dependency for micro-benchmarks.
- **Tested equivalence.** 41 tests (unit, integration, property/fuzz, doctests, λ calibration, Soft-Paging) assert, among other things, SIMD $\equiv$ scalar (randomized fuzz; AVX2, AVX-512, VPOPCNTDQ, NEON), the budget invariants of `enforce_budget` over 300 randomized configurations (`live_bytes` $\leq$ budget, consistent accounting, slot recycling), score finiteness, the Hamming identity, and conformance to Eq. (4). The NEON and VPOPCNTDQ paths are checked by cross-compilation / feature-gated execution. Continuous integration runs `fmt` + `clippy -D warnings` + tests + bench compilation + AArch64 cross-compile.

7 Experimental Evaluation

7.1 Setup and methodological honesty

All numbers below are reproducible with fixed seeds. Measurements use *synthetic* data; the sign-LSH projection Z is *random* by design. In §7.2–§7.3 the low-rank base is assumed ideal ($W_{\text{down}}, W_{\text{up}}$ not learned) and we isolate the INT4-quantization + 1-bit-residual + ranking machinery; $\rho = \|e\|/\|k_{\text{real}}\|$ denotes the energy fraction the base leaves to the residual. §7.4–§7.6 lift the ideal-base assumption by learning the base (PCA, then task-aware SGD). Throughput is reported on a *shared* x86_64 bench (treat as an order of magnitude). We do *not* report real-model perplexity or hardware cache counters; §9 explains why.

7.2 Binary core fidelity (sign-LSH, $d_s = 256$)

Over pairs spanning the full angular range, the 1-bit estimator tracks the true cosine with Spearman > 0.9 (test-proven). In the *realistic* regime, where the residual is near-orthogonal to the query, 256 bits give only moderate resolution: Spearman(residual, $\cos\theta$) ≈ 0.67. This is a genuine limit—one bit per hyperplane resolves nearly orthogonal directions poorly—and motivates larger d_s as future work.

7.3 Full score vs. full-precision ground truth

Table 4 reports rank fidelity of the full SLHA score ($N = 512$ tokens, fixed query) against FP ground truth, as a function of ρ.

Two observations. (i) At low $\rho \leq 0.1$, HOT $\approx$ WARM: freeing the residual is near-lossless when the base captures most of the energy—exactly the condition under which CCOS moves a tile to WARM. (ii) The residual helps everywhere (HOT $\geq$ WARM), with the gap widening as ρ grows (+0.08 Spearman, +12 pts top-16 at $\rho = 0.7$), but the gain is modest at 256 bits.

Table 5: Learned PCA base ($d_{\text{model}} = 256$, latent 128, $d_s = 256$); INT4 with grouped MX microscales.

decay	energy captured	HOT Spearman	HOT top-16	WARM Spearman	WARM top-16
0.99	97.6%	0.788	0.562	0.703	0.438
0.97	99.7%	0.814	0.375	0.700	0.312
0.93	99.5%	0.884	0.438	0.609	0.188
0.85	99.1%	0.905	0.562	0.871	0.500

Table 6: Attention-output fidelity — the closest accessible proxy for perplexity drift.

decay	energy	HOT cos / relL_2	WARM cos / relL_2
0.99	97.6%	0.948 / 0.318	0.943 / 0.333
0.95	99.6%	0.989 / 0.147	0.988 / 0.154
0.90	99.4%	0.994 / 0.108	0.993 / 0.114
0.80	98.9%	0.997 / 0.078	0.996 / 0.082

7.4 Learned PCA base and grouped quantization

Lifting the ideal-base assumption, the base is learned by PCA (best rank-d_c linear reconstruction, Eckart–Young [6]) on synthetic keys with controllable spectrum; Z stays random (Table 5).

HOT plateaus at 0.79–0.90 despite 97–99.7% captured energy: reconstruction energy is not score fidelity. HOT > WARM everywhere, by as much as +0.28 Spearman (decay 0.93). Grouped MX cuts reconstruction error by $> 2\times$ (test-proven) but the end-to-end gain is marginal; whitening the latent *degrades* it (0.859 → 0.750). The plateau is examined in §7.7.

7.5 Attention-output fidelity (the headline result)

Rank fidelity is a proxy; what a model consumes is the attention *output* $\text{out} = \text{softmax}(QK^\top/\sqrt{d})\,V$. Table 6 reports cosine and relative-L_2 between the FP output and the SLHA output (PCA base, $d = 256$, $N = 512$, mean over 64 queries, $d_v = 64$).

This is the most important result. The output is far more robust than the ranking suggested: where score Spearman plateaued at 0.79–0.90, the output cosine reaches 0.95–0.997. The reason is that the softmax averages values, so score errors between tokens of similar weight cancel. HOT $\geq$ WARM throughout (small gap at these high decays, where the residual matters little).

7.6 Task-aware learned projection vs. PCA

PCA picks the highest-variance directions of the *keys* and ignores the *query* distribution. When Q and K favor different subspaces, a projection trained to preserve the *score* $\langle Q, K\rangle$ (not key reconstruction) wins. `train_projection` performs this descent with a closed-form gradient ($a = PQ$, $b = PK$, $r = \langle Q, K\rangle - \langle a, b\rangle$, $\partial r^2/\partial P = -2r(bQ^\top + aK^\top)$), evaluated on an adversarial case (orthonormal factors split into high key-variance / high query-weight halves). We measure in WARM to isolate the projection (Table 7).

Learning beats PCA decisively (WARM 0.86 vs. 0.16): the task-aware projection reallocates low-rank capacity toward the directions that matter for the score. Caveats: synthetic data with deliberate Q/K shift; when Q and K share statistics, PCA is already near-optimal; and this is an SGD proof-of-concept on P alone, not joint training with a real model.

Table 7: Task-aware learned projection vs. PCA under deliberate Q/K shift. SGD score loss $28.8 \to 5.1$.

Projection	WARM Spearman	HOT Spearman	output cos
PCA	0.161	0.325	0.947
Learned	**0.859**	**0.863**	**0.985**

Table 8: Latent codec ablation at decay 0.93.

Latent codec	HOT Spearman	WARM Spearman
INT4 uniform (1 scale)	0.879	0.603
INT4 grouped (MX)	0.884	0.609
NF4 grouped	0.885	0.610
INT8 (ref, 2× bytes)	—	**0.606**

7.7 Ablation: quantization is not the bottleneck

The latent can also be NF4 (normal-float codebook, same 4-bit budget, same 128-byte tile). We add an INT8 reference (coarse only, hypothetical 192-byte tile) to isolate bit-width (Table 8).

NF4 lowers reconstruction error (test-proven) but the end-to-end gain is null-to-marginal ($0.884 \to 0.885$). Crucially, INT8 does *not* beat INT4 on WARM (≈ 0.61): doubling the bits does not lift the coarse ceiling. **The limiting factor is therefore the low-rank projection, not latent quantization.** This corrects an earlier hypothesis that blamed INT4, and refocuses the real levers on (a) a better (learned) projection and (b) the 1-bit residual.

7.8 λ calibration

`calibrate_lambda` compares the residual weight to an FP reference. Writing score = coarse + λr with $r = d_s - 2\,\text{popcount}$ (independent of λ), the optimal multiplier on the formula's λ has the closed form $\alpha^\star = \sum r_t(\lambda r) / \sum(\lambda r)^2$ with $r_t = \langle Q, K\rangle - \text{coarse}$ (Table 9).

The shape $\lambda \propto \sigma_E$ is validated ($\alpha^\star$ constant at 4.18–4.26). The analytic constant is $\approx 4.2\times$ too small; the calibrated constant is $C_{\text{emp}} \approx 0.33$ at $d_s = 256$, which cuts score RMSE by $\sim 15\%$. We expose this value as an opt-in (`LAMBDA_C_CALIBRATED`, with a test asserting its optimal multiplier is ≈ 1), but the crate keeps the analytic form as default for two reasons: the $4.2\times$ factor is measured on synthetic data and the true optimum is model-dependent; and calibration minimizes score *magnitude* (RMSE), a different objective from the *ranking* (top-k/Spearman) that attention ultimately consumes, so over-weighting the directionally-noisy residual could trade ranking for magnitude.

7.9 Throughput (SIMD)

Each kernel path has an equivalence test against the scalar reference (Table 10).

The speedup exceeds the nominal $8\times/16\times$ because the scalar path also paid a branchy INT4 dequant that SIMD fuses. AVX-512 adds only $\sim$23% over AVX2: the kernel is short (128 dims) and bound by nibble-unpacking/loads, not FMA width. In reference cycles (`rdtsc`): $\sim$942 cyc/tile scalar, $\sim$89 AVX2, $\sim$71 AVX-512. A NEON path exists and is cross-compile-verified but not timed (no ARM hardware on this bench).

Table 9: λ calibration. $C_{\text{emp}} = \alpha^\star \cdot \sqrt{\pi/(2d_s)}$.

ρ	$\alpha^\star$ (LS)	C_{emp}	RMSE @formula	RMSE @opt	Δout @formula
0.10	4.18	0.327	1.37	1.26	0.006
0.20	4.23	0.331	2.25	1.94	0.017
0.30	4.24	0.332	3.29	2.79	0.036
0.50	4.25	0.333	5.87	4.92	0.106
0.70	4.26	0.334	9.90	8.26	0.262

Table 10: Score throughput on a shared x86_64 bench (order of magnitude).

Path	Throughput	Ratio
Scalar (reference)	∼2.9 M scores/s	1×
AVX2	∼33.9 M scores/s	**11.5**×
AVX-512 (16-wide FMA/group)	∼41.6 M scores/s	**14.1**×

7.10 Memory traffic vs. a bf16 baseline

`bench_vs_fp16` compares scoring an SLHA tile (128 B/token) against a dot product on a bf16 key ($d_k \cdot 2 = 256$ B/token), both in AVX2 (Table 11).

SLHA scores $\sim 2.5\times$ more tokens/s at comparable memory bandwidth: reading $2\times$ fewer bytes/token (plus a shorter INT4 unpack than bf16 decode) converts directly into throughput—the bandwidth-wall thesis, verified at the kernel level. Throughput decays as context grows (42 → 30 M/s), an indirect cache-residency effect. **Honesty:** the LLC is 260 MB on this bench, so we do not saturate DRAM; the gain comes from byte volume and uops, not a DRAM limit. On a smaller LLC or a genuinely DRAM-bound decode, SLHA's advantage would be larger.

8 Discussion: real levers and false levers

Synthesizing the evaluation:

- **Lever #1 — the low-rank projection.** A learned, task-aware projection beats PCA decisively under Q/K shift (WARM 0.16 → 0.86, §7.6).
- **Lever #2 — the 1-bit residual.** It recovers much of what the coarse term misses (HOT ≫ WARM at high ρ), once λ is calibrated.
- **False lever — latent bit-width.** INT8 does not beat INT4 on the coarse term; NF4 and MX grouping improve reconstruction but barely move end-to-end ranking (§7.7).
- **False lever — SIMD width.** AVX-512 adds only ∼23% over AVX2; the kernel is unpack/load-bound, not FMA-width-bound.

Assumed negative results. Latent whitening degrades fidelity; MX/NF4 grouping yields marginal end-to-end gains; and our own earlier conclusion that INT4 was the bottleneck was refuted by the INT8 measurement and corrected. We report these because they are as informative as the positive results.

Table 11: SLHA (128 B/token) vs. bf16 (256 B/token), AVX2.

Context (tokens)	footprint SLHA / bf16	SLHA	bf16
8 192	1 / 2 MiB	42.4 M/s	17.0 M/s
65 536	8 / 16 MiB	40.1	16.9
262 144	32 / 64 MiB	35.4	14.0
1 048 576	128 / 256 MiB	29.9	13.8

9 Limitations and Threats to Validity

We are explicit about the boundaries of these results.

- **Synthetic data, random Z.** All fidelity numbers use synthetic keys/queries and a random sign-LSH projection. They validate the *mechanism*, not (yet) quality on a real model.
- **No real-model perplexity.** The closest proxy is the attention-output cosine (§7.5); a real perplexity measurement requires a model and dataset not available in our sandbox.
- **No hardware cache counters.** `perf` is unavailable (`perf_event_paranoid=2`); cache effects are shown only indirectly via throughput decay with context size (§7.10).
- **Throughput on a shared bench.** SIMD ratios are an order of magnitude, not a controlled measurement; the LLC was large enough that DRAM was not saturated.
- **λ constant is data-dependent.** The 4.2× correction is measured on synthetic data; the production default stays analytic for that reason.
- **No ARM timing.** The NEON path is compile-checked by cross-compilation only; SLHA's claims for ARM edge targets are therefore architectural, not yet measured.
- **Projection not jointly trained.** The task-aware result optimizes P alone, not end-to-end with a transformer.

10 Future Work

Two of the hardware directions we had identified are now *implemented* and described above: the cache-line-aware alignment (§4.2) and the AVX-512 VPOPCNTDQ residual reduction (§6). The remaining directions are framed as proposals (not measured here).

1. **Scalable SIMD on ARM (Jetson Thor class).** The coarse dot is the hot term; an SVE2 path on Neoverse-V3AE could use *scalable* vectors (adapting to the implementation's vector length) beyond fixed NEON. The residual popcount is already branchless and near-optimal at 256 bits, so the lever is the dot, not the popcount. We deliberately keep this on the roadmap rather than in the shipped kernel for a concrete toolchain reason: SVE2 intrinsics and portable `std::simd` are nightly-gated, whereas the reference crate is intentionally stable and dependency-free. The NEON + `cnt` fallback remains correct meanwhile; SVE2 will land behind runtime vector-length detection once the intrinsics stabilize — and requires ARM hardware to measure.
2. **Host-adaptive alignment via `build.rs`.** The compile-time `cfg_attr` alignment covers cross-compilation to a *known* target. For native builds (`-C target-cpu=native`) a build script could probe the host's *actual* cache-line width and emit a custom `cfg` (e.g. `cache_line_128`), refining

the "AArch64 ⇒ 128" heuristic (not all AArch64 parts use 128-byte lines). This is a portability refinement, not a correctness change.

3. **Salient-outlier tiles (BiLLM-style).** Following BiLLM [10], isolating a few *salient* dimensions in high precision (a `salient_mask` plus a small FP block) while binarizing the rest could raise fidelity within the 128-byte budget. Two non-obvious issues make this a design study rather than a drop-in change. *(i) Metadata budget:* a 16-byte salient block must displace existing metadata, and the natural victims—σ_E and the MX `group_scales`—are exactly the fields that drive CCOS paging (§5) and microscaling, so the trade-off must be measured, not assumed. *(ii) Expected benefit:* our ablation (§7.7) shows quantization is not the current bottleneck on this benchmark; BiLLM targets quantization-error outliers, a regime our synthetic generator does not yet model. The honest prerequisite is therefore to extend the benchmark with realistic activation outliers, so any salient-value gain can be *demonstrated* before it is baked into the tile.

Beyond these: larger d_s to lift the near-orthogonal resolution limit (§7.2); **joint** training of $W_{\text{down}}, W_{\text{up}}$ with a real model (§7.6 establishes the principle); and real hardware validation (`perf` cache counters, end-to-end decode throughput, perplexity).

11 Conclusion

SLHA v2 marries a bit-exact, cache-aware data structure with a hybrid low-rank/1-bit attention score so that the KV cache becomes a fluid, architecture-aware structure rather than a monolithic burden. The mechanism is mathematically correct (test-proven, including scalar/AVX2/AVX-512 equivalence) and directionally validated: HOT ≥ WARM everywhere; Soft-Paging is near-lossless at low residual energy (output cosine ≈ 0.9995 when half the working set is paged); SIMD reaches ∼13×; SLHA scores ∼2.5× more tokens/s than bf16 at the kernel level; a learned projection beats PCA under Q/K shift; and the attention output stays at cosine 0.95–0.997 versus full precision. A controlled ablation reframes the problem: the coarse-score ceiling is set by the low-rank projection, not by latent quantization. What remains is real-hardware and real-model validation. We release the reference implementation, benchmarks, and tests to make every claim reproducible.

Reproducibility. The reference crate (SciRust), all benchmarks, and the 41 tests backing the numbers in §7 are available in the project repository; every measurement uses fixed random seeds.

References

[1] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin. Attention Is All You Need. In *NeurIPS*, 2017.

[2] DeepSeek-AI. DeepSeek-V2: A Strong, Economical, and Efficient Mixture-of-Experts Language Model. arXiv:2405.04434, 2024.

[3] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. E. Gonzalez, H. Zhang, and I. Stoica. Efficient Memory Management for Large Language Model Serving with PagedAttention. In *SOSP*, 2023.

[4] M. S. Charikar. Similarity Estimation Techniques from Rounding Algorithms. In *STOC*, 2002.

[5] W. B. Johnson and J. Lindenstrauss. Extensions of Lipschitz Mappings into a Hilbert Space. *Contemporary Mathematics*, 26:189–206, 1984.

[6] C. Eckart and G. Young. The Approximation of One Matrix by Another of Lower Rank. *Psychometrika*, 1(3):211–218, 1936.

[7] T. Dettmers, A. Pagnoni, A. Holtzman, and L. Zettlemoyer. QLoRA: Efficient Finetuning of Quantized LLMs. In *NeurIPS*, 2023.

[8] E. Frantar, S. Ashkboos, T. Hoefler, and D. Alistarh. GPTQ: Accurate Post-Training Quantization for Generative Pre-trained Transformers. In *ICLR*, 2023.

[9] J. Lin, J. Tang, H. Tang, S. Yang, X. Dang, and S. Han. AWQ: Activation-aware Weight Quantization for LLM Compression and Acceleration. In *MLSys*, 2024.

[10] W. Huang, Y. Liu, H. Qin, Y. Li, S. Zhang, X. Liu, M. Magno, and X. Qi. BiLLM: Pushing the Limit of Post-Training Quantization for LLMs. In *ICML*, 2024.

[11] Z. Liu, J. Yuan, H. Jin, S. Zhong, Z. Xu, V. Braverman, B. Chen, and X. Hu. KIVI: A Tuning-Free Asymmetric 2bit Quantization for KV Cache. In *ICML*, 2024.

[12] Z. Zhang, Y. Sheng, T. Zhou, T. Chen, L. Zheng, R. Cai, Z. Song, Y. Tian, C. Ré, C. Barrett, Z. Wang, and B. Chen. H2O: Heavy-Hitter Oracle for Efficient Generative Inference of Large Language Models. In *NeurIPS*, 2023.

[13] G. Xiao, Y. Tian, B. Chen, S. Han, and M. Lewis. Efficient Streaming Language Models with Attention Sinks. In *ICLR*, 2024.